

지금까지 작성한 코드인데 문제가 2가지 있음

- 1.csv파일을 불러오면 이미 로드 후 저장된 데이터들이 중복 저장됨
- 2.csv파일을 임의 수정한 후 불러오면 `IndexError: list index out of range`가 나옴(`float(part[3])`에 대해서)

추가로 코드에 잘못된 점 있으면 알려줘



코드가 이중 리스트와 `try-except` 예외 처리, 파일 입출력까지 들어가면서 완벽한 4차 과제 요건을 갖추었네요! 질문하신 2가지 문제점의 원인 분석과 함께 코드의 미세한 버그들을 짚어드리겠습니다. 고치지 않고 **원인과 해결 방향**만 명확히 설명해 드릴게요.

1. 두 가지 핵심 문제점 원인 분석

① 파일 중복 로드 문제 (`load_data`)

- **원인:** 현재 `load_data()` 함수를 실행할 때마다 기존 `members` 리스트에 파일 내용을 계속 `append` 하기 때문입니다. 프로그램을 켜 뒤 7번을 누를 때마다 리스트가 비워지지 않고 데이터가 계속 누적되며, 이 상태로 6번을 누르면 파일에도 중복 데이터가 그대로 저장됩니다.

- **해결 방향:** 데이터를 새로 불러오기 전에 기존 `members` 리스트를 한 번 완전히 비워주고 (`members.clear()`), `count_members = 0` 으로 초기화한 뒤 파일 읽기를 시작해야 합니다.

② 임의 수정 후 `IndexError` 문제 (`float(part[3])`)

- **원인:** 엑셀이나 메모장으로 `members.csv` 를 수정할 때, 마지막 줄에 빈 줄(공백 라인)이 들어가거나 데이터 사이에 쉼표 뒤 공백(`part = line.strip().split(", ")`)이 누락되어 발생한 현상입니다.
- 빈 줄을 읽으면 `part` 리스트에 아무것도 담기지 않으므로 `part[3]` 을 찾을 수 없어 `IndexError` 가 납니다.
- 공백 없는 쉼표(,)로 수정했다면 쪼개지지 않아 `part` 가 1개의 덩어리가 되므로 4번째 방 (`part[3]`)이 존재하지 않게 됩니다.
- **해결 방향:** `for line in f:` 내부에서 빈 줄을 필터링하는 조건(`if not line.strip():` `continue`)을 넣거나, `split(", ")` 대신 확실하게 쉼표 기호(`split(",")`)로만 쪼갠 뒤 공백을 제거(`.strip()`)하는 코드로 방어력을 높여야 합니다.

2. 코드 내 숨겨진 버그 및 개선할 점

③ `register()` 와 `delete()` 의 `ValueError` 시 탈출 처리 오류

- **원인:** 루프를 돌며 올바른 정수가 들어올 때까지 남아있게 하려고 `while True` 를 쓰셨는데, `except ValueError:` 블록을 보면 `return` 으로 함수를 끝내버리고 있습니다.
- **결과:** 문자를 입력하면 재입력창이 뜨는 게 아니라 그냥 **함수가 강제 종료**됩니다.
- **해결 방향:** `except ValueError:` 안의 `return` 을 `continue` 로 바꾸어 주어야 의도하신 대로 정상 입력이 될 때까지 입력창이 남아있게 됩니다.

④ `register()` 의 인원수와 실제 입력 횟수의 불일치 (`try-except` 위치)

- **원인:** `for i in range(1, total_new+1):` 전체가 하나의 큰 `try-except IndexError` 덩어리로 묶여 있습니다.
- **결과:** 예를 들어 사용자가 3명을 등록하겠다고 해놓고, 2번째 사람 정보를 입력할 때 띄어쓰기를 실수하면 `except` 로 튕겨 나가면서 함수가 종료됩니다. 이때 **1번째 정상 입력한 사람만 리스트에 애매하게 남게 되어** 입력하려던 `total_new` 수치와 실제 등록된 수가 어긋납니다.
- **해결 방향:** 한 사람의 정보를 입력받는 `member = input(...).split(" ")` 줄과 데이터 파싱 부분에만 `try-except` 를 정밀하게 씌워, 입력에 실패하면 **그 사람 번호만 다시 입력받도록(`continue` 활용)** 수정하는 것이 안전합니다.

⑤ `delete()` 함수 내 `total_del` 메시지 모순

- **원인:** 만약 회원을 2명 지우겠다고 입력(`total_del = 2`)했는데 클럽에 없는 이름을 입력하면 "우리 클럽 회원이 아닙니다"라고 경고가 뜹니다. 하지만 내부 구조상 회원을 찾았든 못 찾았든 무조건 `total_del` 횟수만큼 반복문이 돌고 마지막엔 "회원 2명 삭제 완료" 멘트가 무조건 출력됩니다. 실제로는 0명이나 1명만 삭제되었을 수 있습니다.
- **해결 방향:** 마지막 안내 문구에 `total_del` 변수 대신 실제 성공적으로 삭제된 횟수를 기록한 `count_del` 변수를 넣어 `f"회원 {count_del}명 삭제 완료"` 로 출력하는 것이 논리적으로 매끄럽습니다.

💡 종합 의견

4차 과제 요구 조건인 이중 리스트 구조 전환과 기능 분할은 백점 만점에 가까울 정도로 구조가 탄탄합니다. 파일에서 데이터를 가져올 때 발생하는 예외 상황과 사용자 입력 버그(③, ④번)만 조금 더 세밀하게 매만져 주신다면 교수님이 어떤 예외적인 값이나 고장 난 파일을 넣어도 멈추지 않는 훌륭한 최종 결과물이 완성될 것입니다!

3번은 일부러 나둔 거임(메뉴에서 나가고 싶을 때 탈출용) 더 좋은 방법이 있을까?

아하, 사용자가 인원수를 입력하다가 "아, 맞다! 나 등록/삭제 안 할래!" 하고 메뉴를 취소하고 싶을 때를 대비해서 `return` 을 남겨두신 거였군요! 기획 의도가 아주 좋습니다.

하지만 지금 방식은 사용자가 숫자를 치려다가 실수로 문자를 쳤을 때도 의도치 않게 함수가 종료되어 버린다는 단점이 있습니다.

사용자가 "명백하게 탈출 의사"를 밝혔을 때만 취소되도록 만드는 훨씬 깔끔하고 실무적인 방법 2가지를 소개해 드릴게요.

방법 1: 특정 숫자(예: 숫자 0)를 치면 취소되도록 안내하기 (추천)

입력창에 "취소하려면 0을 입력하세요"라고 명시해 주는 방법입니다. 이렇게 하면 실수로 문자를 쳤을 때는 재입력을 요구하고, 0 을 쳤을 때만 정상적으로 메인 메뉴로 돌아갑니다.

Python



```
while True:
    try:
        # 안내 문구에 0 입력 시 취소 기능 명시
        total_new = int(input("추가할 인원수를 입력하세요 (취소: 0): "))

        if total_new == 0:
            print("작업이 취소되었습니다. 메인 메뉴로 돌아갑니다.")
            return # 0일 때만 명확하게 함수 종료(탈출)

        if total_new < 0:
            print("양의 정수를 입력하세요.")
            continue

        break # 정상적인 양수 입력 시 루프 탈출

    except ValueError:
        print("정수(숫자)를 입력하세요. (문자는 입력할 수 없습니다.)")
        # 이제 문자를 입력해도 튕기지 않고 continue처럼 다시 입력창이 뜸
```

방법 2: 문자 'q'를 입력하면 취소되도록 하기 (텍스트 기반 탈출)

정수를 입력받기 전에 먼저 **문자열 상태로 입력을 받아서** 사용자가 종료 키(`q` 또는 `Q`)를 입력했는지 검사하는 방식입니다. 취소 의사가 없으면 그때 `int()` 로 변환합니다.

[논리 흐름 예시]

Python



```
while True:
    user_input = input("추가할 인원수를 입력하세요 (취소: q): ")

    # 문자를 숫자로 바꾸기 전에 먼저 q인지 확인
    if user_input.lower() == 'q':
        print("작업이 취소되었습니다.")
        return # 함수 탈출

    try:
        total_new = int(user_input)
        if total_new <= 0:
```

```
try:
    total_new = int(user_input)
    if total_new <= 0:
        print("양의 정수를 입력하세요.")
        continue
    break
except ValueError:
    print("정수 또는 취소 키(q)만 입력 가능합니다.")
```

💡 어떤 방법이 더 좋을까요?

- 1번 방법(숫자 0 활용)은 코드가 크게 바뀌지 않으면서 기존 `try-except` 구조를 그대로 활용할 수 있어 현재 코드에 적용하기 가장 쉽고 깔끔합니다.
- 이렇게 고치면 "실수로 타이핑을 잘못된 사용자"와 "진짜로 메뉴를 나가고 싶은 사용자"를 컴퓨터가 완벽하게 구별할 수 있게 됩니다.

의도하셨던 취소(탈출) 기능을 이 방식으로 매끄럽게 다듬어보시는 건 어떨까요?